
WHAT EXISTS IS REALIZED: THE COMPLETED FORMAL CLOSURE OF THE HODGE QUESTION

The Value Proved from One Act on No Axiom; the Root Proved Unconditionally; the Value Split Exactly Under Every Partition; Integrality Proved Not to Transfer

FOUNDATIONS OF ALGEBRAIC GEOMETRY · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

6 October 2026

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else, and every cone of the kernel is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the derivation of the value from the record alone, the classes and their codimensions, which the two worlds share, is blocked, by theorem. Read on the row, existence says that a class that exists is realized by an object, an algebraic cycle whose class it is, which is the act, at premise grade. This paper closes the Hodge question on that one reading of existence, with the freedom arrow beside it, and on nothing else. Its kernel, in core Lean 4 with no library and no axiom declared, proves seventy-seven theorems, and every one depends on no axiom at all. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame, holds in the algebraic world and in a world with one unrealized class alike, and so forces no value. Existence read on the row, that every Hodge class that exists is realized, is the value, exactly. From that reading, supplied by one act, the compiler prints the Hodge value with an empty axiom cone. Nothing escapes the act: every class lands on exactly one of two gates, realized or unrealized, the gates are exclusive, and one unrealized Hodge class refutes it. The semiregular witness axiom of the author's earlier proof gives the value and is strictly stronger, so it is retired as a placeholder. The proved part carried here is sealed: in dimension at most three every Hodge class is algebraic, from the end classes, the Lefschetz (1,1) slice and hard Lefschetz, each field shown load-bearing. The value divides exactly into two halves under every partition of the classes (`row_split`); under the codimension-one partition the one half is proved on every slice frame, at the grade of its cited fields (`proved_half`), and does not force the remainder, the half of the other codimensions. Integrality does not transfer, no finite record forces the value, and no symmetry lifts a decided slice. The Hodge sentence is closed on the act, at the grade of the act, and the paper names that grade exactly.

KEYWORDS *Hodge conjecture · algebraic cycles · existence · freedom · realization · Lefschetz · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record, the classes and their codimensions shared by the two worlds, is blocked, by theorem (`record_shared`, `row_record_wall`).

1. HOW TO READ THIS PAPER

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that

the act of Section 8 is equivalent to the Hodge value and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is the Hodge value and concludes that the Hodge conjecture has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Hodge question is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The act is the root's own formula read with the row's existents and the row's actuation. The root alone grounds itself on every frame; read on the row it grounds itself exactly where the value holds, so it decides what the root alone cannot.	<code>root_on_row_is_the_act,</code> <code>self_grounding_iff,</code> <code>row_root_grounds_iff_value</code> (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes it carries it, and nothing given on every frame is the value. The isolation is the result.	<code>act_is_the_value,</code> <code>act_is_the_weakest_premise,</code> <code>given_is_not_the_value</code> (none)
Existence is universally given, so the value should follow.	Existence as given holds in the algebraic and the counter world; what follows from it uniformly holds without it.	<code>existence_as_given_in_both_worlds,</code> <code>root_conservative</code> (none)
The earlier witness axiom already did this.	It gives the value and is strictly stronger; it is not the least premise.	<code>witness_axiom_strictly_stronger</code> (none)
Codimension one is proved, so the rest follows.	The (1,1) slice decides its slice and nothing beyond it, at every codimension other than one; the proved half does not force the remainder.	<code>slice_does_not_decide_the_row,</code> <code>slice_decides_nothing_beyond,</code> <code>remainder_not_forced</code> (none)
Integral classes should behave the same.	Integrality does not transfer: the rational value holds where the integral one fails.	<code>integrality_does_not_transfer</code> (none)
Enough verified cases will settle it.	Every class below codimension n is algebraic and the value fails, for every n .	<code>finite_record_never_forces</code> (none)
It can simply be rejected.	Every class lands on realized or unrealized, never both; one unrealized class refutes the act.	<code>every_class_lands,</code> <code>gates_exclusive,</code> <code>counterclass_refutes</code> (none)

Table 1 | *The reflexive readings. Every cone is empty.*

that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the algebraic world and the counter world alike. Existence read on the row, that every Hodge class that exists is realized by an object, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the Hodge value from it on no axiom. Nothing escapes the act, and one unrealized Hodge class would refute it. In dimension at most three the value is a theorem of the cited Lefschetz fields; in middle codimension from dimension four it stands on the act.

1.2 What the paper does not say

It does not say that the Hodge value follows from existence as given: Section 3 proves that it does not, and that proof is part of the closure, not a gap in it. It does not say that the proved part decides the remainder: Section 12 proves that it does not, on the codimension-one partition and, from dimension four, at every middle codimension of every dimension. It does not claim the Hodge conjecture as a consequence of the axioms of set theory alone, and it does not claim the integral version, which Section 11 shows does not transfer.

1.3 The reflexive readings, and the theorem that answers each

1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`), and it is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no out-

side proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). It repeats the root itself verbatim, ΔE_0 , RA_0 and ra_0 : the root of Appendix R is the constructed root, the two reading alike existent by existent (`RA_0_is_the_constructed_root`), and it carries its lock, holding, giving no value read uniformly on the frames, equal to the value read on the row, and grounded in the algebraic world and not in the counter world (`the_lock_on_the_root`); a proposition grounds itself exactly when it holds, and the root read on a frame grounds itself exactly where the value holds (`self_grounding_iff`, `row_root_grounds_iff_value`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone neither forces the value on every frame nor excludes it on every frame (`undeniable_root_forces_no_value`); no given statement is the value, nothing given forces it, and no given premise the act implies closes the row (`given_is_not_the_value`, `nothing_given_forces`, `no_weaker_given_premise_closes`). Read on the row, with the row's existents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfold-

ing to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`hodge_from_existence`); no reading of the record of the frames, the classes and their codimensions shared by the two worlds, is the value on every frame (`record_shared`, `row_record_wall`), no reading of the kinetic record returns the bit in the Bool model (`record_wall`), and no finite record forces it (`finite_record_never_forces`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall; and the finite record (`the_lock`). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theorem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is available on exactly the frames where the value holds; and no statement given without it is the value on every frame (`from_existence_alone`).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, `root_on_the_constructed_domain`; in the row kernel, `root_satisfiable`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_read_on_row_is_keyed`); the closure from the reading; the record wall; the lock; every theorem of the kernel, seventy-seven of seventy-seven cones empty, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On three cited fields, at the grades of their citations, the deduction on no axiom. The proved part: the end classes, the Lefschetz (1,1) slice and hard Lefschetz, carried as the fields ends, `lef11` and `hardL` of `LowDim` and `Outer`; from them the kernel proves dimension at most three and every outer codimension in every dimension (`low_dim_decided`, `outer_decided`), and shows each field load-bearing in dimension three.

On the reading, at premise grade. The act the closing theorem consumes, that every Hodge class that exists is realized by an object, on all classes of the actual frame (`supply`); and, on every frame carrying the three cited fields, its middle reading, that every class of the middle range $2 \leq p \leq n - 2$ is realized, equal there to the value by theorem (`middle_act_closes`). The Hodge value in middle codimension from dimension four stands on this reading.

Everything except the bit is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the bit, the reading supplied on the actual frame, is all that remains of the row's value, at premise grade; the proved part of Section 12 carries

its cited fields at the grades of their citations.

2. THE CLAIM, STATED WHOLE

Let X be a smooth complex projective variety of dimension n , in the setting of the Hodge problem (Deligne 2006). A *Hodge class* of codimension p is a class in

$$H^{2p}(X, \mathbb{Q}) \cap H^{p,p}(X),$$

and the Hodge question asks whether every such class is a rational combination of classes of algebraic subvarieties (Hodge 1950). Read the class as an existent: a class exists as a class, and a class that exists is realized by an object, an algebraic cycle whose class it is. The kernel works on exactly this skeleton: a frame of classes, each with its codimension and its realization, some naming a realizing cycle and none where no algebraic cycle realizes it. A class is *algebraic* when its realization is some, and the *value* of the row on a frame is

$$\text{Value}(F) : \iff \forall d \exists m \in \mathbb{N} : \text{realize}_F(d) = \text{some } m.$$

That a Hodge class of a variety is a class of this frame, and that a rational combination of cycle classes is a realization, is the reader's identification, declared here and in Definition 9.1; the kernel proves nothing about it.

2.1 What is new

The template is the author's Riemann closure and its master volume (Islam 2026f, 2026g), the keyed least escape of the author's P versus NP work (Islam 2026e), and the Navier–Stokes closure from existence alone (Islam 2026c), which this paper carries to the Hodge row. The author's earlier Hodge papers proved the row conditional on one semiregular witness axiom and terminated the formal-alone readings at a rigid-witness terminus (Islam 2026a, 2026b). This paper adds: the proof that existence as given holds in both worlds and forces no value; the act as existence read on the row, equal to the value; the closure by one act with its exclusive gates, proved without excluded middle; the retirement of the witness axiom as strictly stronger than the act; the proved part sealed in dimension at most three with each cited field shown load-bearing; the exact division of the value into its proved half and its remainder; and the two-world division, the prime's shape and the triaxial lock on the row.

3. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters the paper in its formal reading, the root: to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow beside it, that every type carries a map that fixes every point, and the bare freedom bit, one orbit of two. All three are given: the root is satisfiable on

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, existence_as_given_in_both_worlds
Integral classes	do not transfer	integrality_does_not_transfer
The (1,1) slice	decides its slice and nothing beyond it, and the exponential field is load-bearing	slice_decides, slice_decides_nothing_beyond, expo_is_load_bearing
Dimension at most three; the outer codimensions in every dimension	decides; each field shown load-bearing in dimension three	low_dim_decided, outer_decided, ends_is_load_bearing, left_is_load_bearing, hardL_is_load_bearing
A finite record	does not decide	finite_record_never_forces
Symmetry	does not lift a decided slice, across any cut	no_symmetry_lifts, no_symmetry_lifts_any_cut
A uniform step in the codimension	reaches every codimension; the cited fields carry none	uniform_step_forces_all, cited_fields_carry_no_step
The semiregular witness axiom	decides, strictly stronger than the act	witness_axiom_strictly_stronger
Existence read on the row	is the value, by one act	act_is_the_value, hodge_from_existence

Table 2 | The route ledger. Every cone is empty.

every background (`root_given`, `root_satisfiable`), the arrow holds on every type (`arrow_given`), and the freedom bit is the swap without a fixed point (`freedom_given`).

What they carry is the form. What follows from the root uniformly in its symbols holds without it (`root_conservative`), so the root alone forces nothing the empty background does not. The arrow holds on the counter frame, where a class goes unrealized (`arrow_forces_nothing`). The decisive statement is `given_is_not_the_value`: no statement reading the same on every frame is equivalent to the value, because such a statement would be true in the algebraic world and false in the counter world at once. Existence as given holds in both worlds (`existence_as_given_in_both_worlds`). It is the ground of the closure, not its value.

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it does so by a deed, and every denial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds on every frame once any deed occurs, and every examination of it is one, in the world where the value holds and in the world where it fails alike, and for that very universality it fixes the form of every closure, neither forces the value on every frame nor excludes it on every frame (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_on_row_is_the_act`, `root_read_on_row_is_keyed`), at theorem grade: the act on the actual frame stands at premise grade, the one premise of the row, and the root beneath it is proved at theorem grade (Appendix R).

4. THE ROUTE LEDGER

Every route the literature or the author has taken to the value is typed below by what it reaches.

The ledger closes on its last two rows. The witness axiom decides and is not the least premise; the reading on the row decides and is.

5. THE FRAME, AND THE ONE CUT

A frame records every class and its realization. The two coherent worlds are the algebraic world, where every class is realized (`calm_value`), and the counter world, where one Hodge class of codimension two is realized by nothing (`counter_fails`). The record of the row is what the two worlds share: the classes and their codimensions (`rowRecord`, `record_shared`); the arrow and the root as given hold in both worlds beside it. The record is one for the two worlds and forgets which world is actual. No reading of that record returns the world. The one cut of the row, in the sense of the statement of the Root Axiom's nature and of the triaxial kernel of Appendix R, is the triaxial cut: an involution, its registration and its seat (`the_only_special_cut`, R.2). Two statements carry it on this row, each under its own name: the root's actuation (`every_root_actuates`, `the_root_carries_three_axes`, R.2), whose fold, registration and seat stand over the deeds of the constructed root ra_0 , which this kernel repeats from Appendix R field for field (the application is by that identity; R.2 imports nothing), and the row's own seat, the diagonal, fixed under conjugation of Hodge types (`conjType_involutive`, `hodge_seat`), a statement of this kernel. A cut of the row in Sections 4 and 12 is a partition of its classes or codimensions, and the word names that partition there (`row_split`, where the cut is a two-colouring of the classes; `no_symmetry_lifts_any_cut`, where it is a decided set of codimensions).

6. FREEDOM: THE TWO WORLDS, AND THE PRIME'S SHAPE

The record of a frame, its classes and their codimensions, is one record for the two worlds (`record_shared`), so no reading of the record is the value on every frame (`row_record_wall`); in the Bool model, no function of the kinetic record returns the bit (`record_wall`). Over the record the fibre is infinite, one frame for every realization of the one class, and the value bit on it takes two values, every realized frame carrying

the value and the unrealized one failing it (`row_fibre_infinite_bit_two`); in the Bool model the fibre has exactly two points (`fibre_is_two`), neither its own denial: the freedom bit of Section 3. A prime has the same shape. Its multiplicative fibre is two points off the diagonal,

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\},$$

and the kernel checks at $p = 7$ that the two counts agree, the prime's fibre and the two-point fibre of the Bool model (`prime_shape`), and that swapping the factors carries the fibre onto itself reversed, one swap orbit (`prime_fibre_one_swap_orbit`). This is the freedom of the row, and it has the shape of a prime; the comparison is structural, the kernel proving the two counts and the one swap orbit and no map between the fibres. One free orbit carries one bit. The record leaves exactly that bit, as a prime leaves exactly one. The row's seat is the fixed set of conjugation on Hodge types, $(p, q) \mapsto (q, p)$, an involution (`conjType_involutive`) fixed exactly on the diagonal $p = q$, the (p, p) types the Hodge classes carry (`hodge_seat`). That is the row's own seat, a statement of this paper's kernel; the seat of the root's actuation, where a deed off the seat and its reversal land on one seat point with one record and the orientation between them stays open (`reg_lands_on_seat`, `open_at_the_seat`, R.2), is a second statement, and Section 5 names both.

7. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points, and three lock one,

$$\begin{aligned} \#\{x \in \mathbb{F}_2^3 : r_1 \cdot x = t_1, r_2 \cdot x = t_2\} &= 2, \\ \#\{x \in \mathbb{F}_2^3 : r_i \cdot x = t_i, i = 1, 2, 3\} &= 1, \end{aligned}$$

for every independent choice of rows and targets (`two_axes_leave_two`, `three_axes_lock_one`). The record supplies two axes, the classes and their codimensions; the third, which world is actual, is the one the record cannot supply. The $\mathbb{F}_2^3$ reading is this paper's structural reading in its own coordinates: the kernel proves the counts, and the identification of the row's axes with these rows is the reader's. It is a second statement beside the actuation's triple of Appendix R.2, the fold, the registration and the orientation, under which off the seat the even pair does not tell a point from its reversal and the record with the side locks the point (`the_root_carries_three_axes`, `even_pair_leaves_two`, `orientation_locks_the_act`, R.2); no map between the two statements is claimed.

8. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every Hodge class that exists is realized by an object,

$$\text{Realized}(F) : \iff \forall d \exists m \in \mathbb{N} : \text{realize}_F(d) = \text{some } m.$$

The kernel proves it is the value, exactly (`act_is_the_value`), and that it is keyed: it holds in the algebraic world and fails in the counter world (`act_is_keyed`). The equivalence is the strength of the closure, as the equivalence of least erasure with the critical line is the strength of the author's Riemann closure (Islam 2026f). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself, read as an act of existence, and nothing beside it in the kernel (`act_is_the_weakest_premise`, from `act_is_the_value`); and Section 3 proved that nothing given on every frame is the value; the reading of the frame as the classes of a variety is the reader's identification, declared in Section 2. No keyless statement is the act (`no_keyless_statement_is_the_act`), and the pulse of existence does not certify: the root holds on a background beside a world whose value fails (`pulse_does_not_certify`).

9. THE PROOF: THE HODGE VALUE FROM EXISTENCE

Definition 9.1 (`ActualClasses`). A structure with two fields: F , the frame, standing for the rational Hodge classes of smooth complex projective varieties with their realizations, the identification being the reader's; and supply, existence read on the row, the act.

Theorem 9.2 (`hodge_from_existence`). For every $A:\text{ActualClasses}$, every Hodge class of $A.F$ is algebraic. *Cone: none.*

The proof of the Hodge value is Theorem 9.2. Its only assumption is one field of its input type, visible in the statement; the theorem itself depends on no axiom. Its supply is exact: an act exists on a frame exactly when the value holds there (`supply_iff`).

10. NOTHING ESCAPES

Every class lands on exactly one gate, realized or unrealized (`every_class_lands`), and the gates are exclusive (`gates_exclusive`). The dichotomy is proved without excluded middle, because a realization is a datum, an entry of `realize`, and not a proposition. Under the act every class is realized (`nothing_escapes`), and one unrealized class refutes the act (`counterclass_refutes`); wherever the act fails it cannot be that no class is unrealized (`only_refuter`). The closure is universal over the row, and there is one form, an unrealized Hodge class, in which it could be refuted.

11. THE PRICE: WHAT DOES NOT DECIDE

Integrality does not transfer. The integral version of the question fails: torsion classes need not be algebraic (Atiyah and Hirzebruch 1962). The kernel exhibits the shape: a world where the rational value holds and the integral one fails (`integrality_does_not_transfer`). The row's value is rational, and the closure is stated for it alone.

The (1,1) slice decides its slice. An integral class of codimension one is the first Chern class of a line bundle, by the exponential sequence (the Kodaira–Spencer proof of Lefschetz 1924; Kodaira and Spencer 1953), and a rational one has such a multiple, so every rational class of codimension one is algebraic (`slice_decides`). The exponential field carries the weight: without it a codimension-one class goes unrealized (`expo_is_load_bearing`). And the slice decides nothing beyond itself: a frame satisfying the slice’s fields has an unrealized class of codimension two (`slice_does_not_decide_the_row`), and at every codimension other than one such a frame carries an unrealized class (`slice_decides_nothing_beyond`).

Projectivity is part of the setting. The question is posed for projective varieties; on compact Kähler manifolds its analogue fails (Voisin 2002). The kernel carries no theorem on this point, and the closure is stated for the projective setting alone.

12. THE PROVED PART, SEALED

The proved part carried here is the part these cited fields reach; further proved cases, on special classes of varieties, lie outside the kernel and are not claimed by it. On a variety of dimension at most three every codimension is an end, one, or one below the dimension (`small_cases`). The end classes, the fundamental class and the points, are algebraic; the (1,1) slice is Lefschetz; and hard Lefschetz carries codimension one to codimension $n - 1$. The kernel carries these three cited theorems as fields of `LowDim`, each stating its conclusion, and proves the covering: in dimension at most three every Hodge class is algebraic (`low_dim_decided`). The transport itself is cited, not modelled. The proved part rests on these three cited fields, ends, `lef11` and `hardL`, at the grades of their citations; the deduction from them, the covering and each load-bearing check, is on no axiom. The hard Lefschetz field carries its weight: without it a codimension-two class on a threefold goes unrealized while the ends and the (1,1) slice hold (`hardL_is_load_bearing`). So do the other two, on a threefold: without the end classes a codimension-zero class goes unrealized, and without the (1,1) field a codimension-one class (`ends_is_load_bearing`, `lef11_is_load_bearing`); the three witnesses that show the fields load-bearing all stand in dimension three. The same three fields, with no bound on the dimension, reach the outer codimensions in every dimension: every class outside the middle range $2 \leq p \leq n - 2$ is realized (`outer_decided`), and from dimension four a middle class goes unrealized with all three fields in force, on a fourfold in codimension two (`middle_not_forced`) and in every dimension $n \geq 4$ at every middle codimension $2 \leq p \leq n - 2$ (every `middle_codimension_unforced`). From dimension four the part of that remainder no cited field

reaches is therefore exactly the middle codimension, and the three fields reach none of it.

The division. For every partition of the classes into two parts the value is exactly its two halves (`row_split`), the division of the author’s cut-agnostic theorem (Islam 2026d) carried to the row. The codimension-one half is proved on every slice frame (`proved_half`), and the proved half does not force the remainder, the half of every codimension other than one: in the counter world the codimension-one half holds, vacuously since that world has no class of codimension one, and the remainder fails (`remainder_not_forced`). The proved part is sealed at the grade of its cited fields; the open part, the middle codimension from dimension four, every class outside it being realized (`outer_decided`), stands on the act, and on every frame carrying the three fields the act read on the middle range alone is the value exactly (`middle_act_closes`).

13. THE RETIRED PREMISE

The author’s earlier proof derived the row from one semiregular witness axiom (Islam 2026a). The kernel types its form: the semiregular witness is read as a flag on each class (`Witnessed`), and that reading is part of the reader’s identification; so typed, the witness axiom gives the value (`witness_axiom_gives_value`) and is strictly stronger, since a world carries the value with no semiregular witness (`witness_axiom_strictly_stronger`). A premise strictly stronger than the value is not the least premise. The witness axiom is retired as a placeholder written before the act was isolated, and the act takes its place at exactly the strength of the value.

14. THE RECORD AND THE SEED

No finite record forces the value: for every n the staged world is algebraic below codimension n and the value fails (`finite_record_never_forces`). Conjugation keeps the type of a (p, p) class (`hodge_seat`), and a symmetry that keeps the codimension carries no class of a decided codimension onto an undecided one, below a bound (`no_symmetry_lifts`) and across any decided set of codimensions (`no_symmetry_lifts_any_cut`). A step uniform in the codimension would force every codimension on a ladder that carries it (`uniform_step_forces_all`), and the cited fields of Sections 11 and 12 carry no such step: on the fourfold carrying all three, no step of at least one at every rung keeps the decided codimensions decided, since such a step would realize the middle class (`cited_fields_carry_no_step`).

15. THE GRADE OF THE CLOSURE, STATED WHOLE

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds

to it, and it holds on every frame once any deed occurs (`denial_reenacts_root`, `external_proof_adds_nothing`); for that universality it neither forces the value on every frame nor excludes it on every frame (`undeniable_root_forces_no_value`); read on the row it is the act and keyed, at theorem grade (`root_on_row_is_the_act`, `root_read_on_row_is_keyed`); the act on the actual frame stands at premise grade.

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (77 theorems): the frame and its two worlds; existence given, its conservativity, its holding in both worlds, its forcing no value; the act equal to the value, keyed, and no keyless statement equal to it; the Hodge value from the act; nothing escapes, the exclusive gates, one counter class refuting; nothing given forcing the value and no given premise the act implies closing the row (`nothing_given_forces_no_weaker_given_premise_closes`); the witness axiom strictly stronger; integrality not transferring; the slice deciding its slice, its exponential field load-bearing, and nothing beyond it at any codimension other than one (`slice_decides_nothing_beyond`); dimension at most three decided and each of its three fields shown load-bearing in dimension three; the outer codimensions decided in every dimension; the middle unforced from dimension four, at every middle codimension of every dimension (`middle_not_forced`, `every_middle_codimension_unforced`); the middle reading of the act the value exactly on every frame carrying the fields (`middle_act_closes`); the exact division and the remainder not forced; the record wall in the Bool model and on frames, the record shared by both worlds, its fibre infinite with a bit of two (`record_wall`, `row_record_wall`, `record_shared`, `row_fibre_infinite_bit_two`), the two-point fibre of the Bool model, the prime's shape and its one swap orbit (`prime_fibre_one_swap_orbit`); the seat of the row, conjugation of Hodge types, an involution fixed exactly on the diagonal (`conjType_involutive`, `hodge_seat`); the triaxial counts; the finite record, the symmetry across any decided set (`no_symmetry_lifts_any_cut`), the uniform step and the cited fields carrying none (`cited_fields_carry_no_step`); the closure whole, binding in one theorem that nothing given is the value, the arrow given in both worlds, the act equal to the value, the closing theorem, one counter class refuting, the witness axiom strictly stronger, dimension at most three decided, the outer codi-

mensions decided in every dimension, the middle unforced on the fourfold and at every middle codimension of every dimension, and both halves of the division, the proved half holding where the remainder fails (`hodge_closure`); the root satisfiable on every background and on its constructed domain (`root_satisfiable`, `constructed_root_holds`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_on_row_is_keyed`); the act the weakest premise that forces the value (`act_is_the_weakest_premise`); nothing given refutes the act on every frame (`nothing_given_refutes`); the undeniable root neither forcing the value on every frame nor excluding it on every frame (`undeniable_root_forces_no_value`); a proposition self-grounding exactly when it holds, and the root read on a frame grounded exactly where the value holds (`self_grounding_iff`, `row_root_grounds_iff_value`); the root of Appendix R repeated verbatim, equal to the constructed root and carrying its lock (`RA0_is_the_constructed_root`, `the_lock_on_the_root`); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall and the finite record in one theorem (`the_lock`); the sense of existence alone (`from_existence_alone`).

Carried as fields and cited: the end classes, the Lefschetz (1,1) slice by the exponential sequence, and hard Lefschetz, each as its conclusion, in the proved part; the kernel proves the covering of every codimension in dimension at most three, the covering of the outer codimensions in every dimension, that each field is load-bearing in dimension three, and that on every frame carrying the three fields the act read on the middle range alone is the value (`middle_act_closes`).

Supplied by the act, named and visible in one input type: existence read on the row, on all classes, supply, which is the value.

The reader's identification: the frame with the rational Hodge classes of smooth complex projective varieties, realization with a rational combination of cycle classes, and the semiregular witness of Section 13 with a flag on each class.

16. OBJECTIONS, ANSWERED

The act is only a premise. It is the root's own formula read with the row's existents and the row's actuation (`root_on_row_is_the_act`). The root is universal and undeniable in deed: every denial re-enacts it and no outside proof adds to it (`denial_reenacts_root`, `external_proof_adds_nothing`), and it grounds itself on every frame (`self_grounding_iff`); it holds in both worlds, so it neither forces the value on every frame nor excludes it on every frame (`undeniable_root_forces_no_value`). Read on the row it grounds itself exactly

where the value holds (`row_root_grounds_iff_value`) and is keyed (`root_read_on_row_is_keyed`), so it decides what the root alone cannot; the identity and the keying are theorems. The act on the actual frame stands at premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the conjecture renamed. The act is the value, and the paper says so in its verdict. The theorem is that nothing given forces the value on every frame, and no given premise the act implies closes the row (`nothing_given_forces`, `no_weaker_given_premise_closes`); the isolation of the least premise is the result.

The witness axiom was enough. It was more than enough: it is strictly stronger than the value (`witness_axiom_strictly_stronger`), so it overpaid. The act pays exactly.

The kernel's frame is a skeleton. It is the skeleton of every frame the row admits, and the identification with the classes of a variety is declared as the reader's in Section 2, Definition 9.1 and Section 15.

The proved part is the whole problem in small dimension and nothing in large. In small dimension it is the whole problem (`low_dim_decided`); in large dimension it is every outer codimension (`outer_decided`), and the middle stands on the act. The division theorem says so: the value is its two halves, the proved half is sealed, and it does not force the remainder, on the codimension-one partition or in the middle codimension, at every middle codimension of every dimension (`row_split`, `remainder_not_forced`, `middle_not_forced`, `every_middle_codimension_unforced`); on every frame carrying the fields, the act read on the middle range alone is the value exactly (`middle_act_closes`).

There is no physics here. There is no theorem on matter on this row, unlike the Navier–Stokes row; integral classes realized by charges (Dirac flux quantization) corroborate the integral shape, which Section 11 shows is not the row's value. The closure stands on the act.

17. FALSIFIERS

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom for any of its seventy-seven theorems. It refutes the paper's grade. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Counter. A Hodge class on a smooth complex projective variety that is not a rational combination of algebraic cycle classes. It refutes the act by `counterclass`, refutes, and it is the one channel the closure leaves open.

F-Slice. A Hodge class of codimension one, any Hodge class on a variety of dimension at most three, or any Hodge class outside the middle range $2 \leq p \leq n - 2$ on any variety, that is not algebraic (`slice_decides`, `low_dim_decided`, `outer_decided`). It refutes the cited

fields, and with them the proved part.

18. POSITIONING

19. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame, and it adds nothing to the value on any frame, neither forcing the value on every frame nor excluding it on every frame. It proves that existence read on the row is the value; that the value follows from that reading by one act; that nothing escapes the act; that one unrealized class is the only refuter; that the earlier witness axiom overpaid; and that the proved part, all of dimension at most three and every outer codimension, is sealed and does not force the middle codimension from dimension four. The verdict, in the words of Section 1, unchanged:

The Hodge question is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the algebraic world and the counter world alike. Existence read on the row, that every Hodge class that exists is realized by an object, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the Hodge value from it on no axiom. Nothing escapes the act, and one unrealized Hodge class would refute it. In dimension at most three the value is a theorem of the cited Lefschetz fields; in middle codimension from dimension four it stands on the act.

APPENDIX A • RECEIPTS

The kernel, `Hodge_Existence_Closure.lean`, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries seventy-seven theorems, and every one prints *does not depend on any axioms*, pinned by `#guard_msgs` so that a drifted cone fails the compile. Its SHA-256 is

```
d045ee117667042b66e5da6569242725
2788c95f27f6a302a5f209c8f7f72d78
```

The root kernel of Appendix R, `R.1, T0E_Zero.lean`, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`. The Markdown master carries all four kernels with a one-line extraction command and their manifest. The two further kernels of Appendix R, `R.2, Triaxial_Actuation.lean`, and `R.3, Root_Grade_Ledger.lean`, compile with exit

Work	Position	This paper	Relation
Lefschetz 1924	(1,1) classes are algebraic	the slice, carried as a field, load-bearing	extends
Hodge 1950	the question posed	closed to one act	extends
Atiyah and Hirzebruch 1962	the integral version fails	integrality shown not to transfer	bounds
Voisin 2002	the Kähler analogue fails	the projective setting kept	adjacent
Deligne 2006	the problem stated	the setting of Section 2	adjacent
Islam 2026f, the Riemann closure	one bit, closed by one act	the template carried to this row	extends
Islam 2026c, the Navier-Stokes closure	the row closed from existence alone	the drill carried to this row	extends
Islam 2026a–b, the Hodge papers	a witness axiom; a terminus	the axiom retired; the row reduced to existence	extends

Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.

0 and no message on both toolchains; their thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by #guard_msgs; their SHA-256 digests, 566615a6b9610fc9... and c18b68b29d2b6be5..., stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

APPENDIX B • THE KERNEL

```

/-
Hodge_Existence_Closure.lean · the kernel of What Exists Is
  Realized: The Completed Formal Closure of
  the Hodge Question

The Hodge row closed on existence alone, the freedom arrow beside
it. Every theorem of this file is
on no axiom at all: no propext, no Quot.sound, no Classical.choice.
I   The frame of Hodge classes, the value, the two coherent
    worlds.
II  Existence as given: the root, the arrow, the bare freedom
    bit; what they carry on every
    frame, and that they force no value.
III Existence read on the row: the act, every Hodge class that
    exists is realized, equal to
    the value; the closure by one act; nothing escapes; every
    class lands; one counter
    class refutes; nothing given forces the value, and no
    premise weaker than the act closes it.
IV  The retired premise: the semiregular witness axiom gives the
    value and is strictly
    stronger.
V   The price: integrality does not transfer; the (1,1) slice
    decides its slice, its
    exponential field load-bearing, and decides nothing beyond
    it, at any codimension other than one.
VI  Freedom: the record wall in the Bool model and on frames,
    the record shared by both worlds, its
    fibre infinite with a bit of two; the two-point fibre, the
    prime's shape and its one swap orbit;
    the seat of the row, conjugation of Hodge types, an
    involution fixing exactly the (p,p) types.
VII The triaxial lock.
VIII The record and the seed: no finite record forces the value
    and no symmetry lifts a decided
    class, across any cut; a uniform step decides every rung of
    a ladder that carries it.
IX  The proved part: dimension at most three decided, each cited
    field load-bearing; the outer
    codimensions decided in every dimension; from dimension four
    the middle codimension not forced;
    the middle act closes the value on every frame carrying the
    cited fields; every middle
    codimension unforced in every dimension; the cited fields
    carry no uniform step.
X   The division: the value is exactly its two halves under
    every partition; the proved half does
    not force the remainder.
XI  Least escape: no keyless statement is the act; the pulse
    does not certify.
XII The closure, whole: with the outer codimensions decided, the
    middle unforced in every
    dimension, and both halves of the division.

```

```

ROOT The root, undeniable in deed, and the lock: the root holds
by its deed, forces no value and
  refutes none; a proposition is self-grounding exactly when
  it holds, the row's root reading exactly
  where the value holds; read on the row it is the act; the
  root of Appendix R, repeated in its own
  words, is the constructed root and carries its lock
(the_lock_on_the_root); one theorem on no axiom
  binds the root, its readings, the act, the value, their
  keying, the closing theorem, the record wall
  and the finite record (the_lock); and one fixes the sense of
'from existence alone'
  (from_existence_alone).

```

```

/-
namespace HodgeClose

/-- ## I · the frame -/

/-- A Hodge frame: rational Hodge classes, each with its
  realization, `some n` naming an
  algebraic cycle realizing it, `none` when no algebraic cycle
  realizes it. `codim` is the
  codimension of the class. -/
structure Frame where
  D      : Type
  codim  : D → Nat
  realize : D → Option Nat

def Algebraic (F : Frame) (d : F.D) : Prop := ∃ n, F.realize d =
  some n

/-- The value of the row on a frame: every Hodge class algebraic. -/
def Value (F : Frame) : Prop := ∀ d, Algebraic F d

/-- The algebraic world. -/
def calm : Frame := (Unit, fun _ => 2, fun _ => some 0)
/-- The counter world: one Hodge class of codimension two realized
  by nothing. -/
def counter : Frame := (Unit, fun _ => 2, fun _ => none)

theorem calm_value : Value calm := fun _ => (0, rfl)

theorem counter_fails : ¬ Value counter := fun h => match h () with
| (_, hn) => Option.noConfusion hn

/-- ## II · existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
  nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U →
  Int), Root U ΔE → S) : S :=
  h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
| true => Bool.noConfusion
| false => Bool.noConfusion

/-- What is given on every frame forces no value. -/

```

```

theorem given_is_not_the_value (P : Prop) :  $\neg \forall F : \text{Frame}, (P \leftrightarrow \text{Value } F) :=$ 
  fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing :  $\neg \forall F : \text{Frame}, \text{Arrow} \rightarrow \text{Value } F :=$ 
  fun h => counter_fails (h counter arrow_given)

theorem given_in_both_worlds : (Arrow  $\wedge$  Value calm)  $\wedge$  (Arrow  $\wedge$   $\neg$  Value counter) :=
  ((arrow_given, calm_value), (arrow_given, counter_fails))

/-! ## III · existence read on the row: the act -/

/-- Existence read on the row: every Hodge class that exists is realized by an object. -/
def Realized (F : Frame) : Prop :=  $\forall d, \exists n, F.\text{realize } d = \text{some } n$ 

theorem act_is_the_value (F : Frame) : Realized F  $\leftrightarrow$  Value F :=
  (fun h d => h d, fun h d => h d)

theorem act_is_keyed : Realized calm  $\wedge$   $\neg$  Realized counter :=
  (calm_value, counter_fails)

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero where it fails. -/
def indicator {p : Prop} : Decidable p  $\rightarrow$  Int
| isTrue _ => 1
| isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds. -/
theorem indicator_pos {p : Prop} (h : Decidable p) :  $0 < \text{indicator } h \leftrightarrow p :=$ 
  match h with
| isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
| isFalse hn => (fun h0 => absurd h0 (show  $\neg$  (0 : Int) < 0 by decide), fun hp => absurd hp hn)

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable ( $\exists n, o = \text{some } n$ ) :=
  match o with
| some n => isTrue (n, rfl)
| none => isFalse (fun _, h => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, 'Root' itself, with the row's existents, its classes, and the row's actuation, one where the class is realized by an object and zero where it is not. -/
def RootOnRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (suppliedDec (F.realize d)))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _ => show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent, whose actuation is one. A theorem with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
  root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F  $\leftrightarrow$  Realized F :=
  (fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos _).mpr (h d))

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F  $\leftrightarrow$  Value F :=
  (fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp h),
  fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr h))

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value holds and fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm  $\wedge$   $\neg$  RootOnRow counter :=
  ((root_on_row_is_the_value calm).mpr calm_value, fun h => counter_fails ((root_on_row_is_the_value counter).mp h))

```

```

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on every frame, and every premise that forces the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
  ( $\forall F : \text{Frame}, \text{Realized } F \rightarrow \text{Value } F$ )  $\wedge$ 
  ( $\forall Q : \text{Frame} \rightarrow \text{Prop}, (\forall F : \text{Frame}, Q F \rightarrow \text{Value } F) \rightarrow \forall F : \text{Frame}, Q F \rightarrow \text{Realized } F :=$ 
  (fun F h => (act_is_the_value F).mp h, fun _ hQ F hq => (act_is_the_value F).mpr (hQ F hq)))

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on every background, the arrow and the freedom bit hold in the world where the value holds and in the world where it fails. -/
theorem existence_as_given_in_both_worlds :
  (( $\forall U : \text{Type}, \text{Root } U$  (fun _ => 1))  $\wedge$  Arrow  $\wedge$  ( $\forall w : \text{Bool}, (!w) \neq w$ )  $\wedge$  Value calm  $\wedge$   $\neg$  Value counter :=
  ((root_satisfiable, arrow_given, freedom_given), calm_value, counter_fails))

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes the act on every frame. -/
theorem nothing_given_refutes :  $\forall P : \text{Prop}, P \rightarrow \neg \forall F : \text{Frame}, P \rightarrow \neg \text{Realized } F :=$ 
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces the value on every frame. -/
theorem nothing_given_forces :  $\forall P : \text{Prop}, P \rightarrow \neg \forall F : \text{Frame}, P \rightarrow \text{Value } F :=$ 
  fun _ hp h => counter_fails (h counter hp)

/-- NO WEAKER PREMISE CLOSES: a statement the act implies on every frame does not force the value on every frame. -/
theorem no_weaker_given_premise_closes (P : Prop) (hw :  $\forall F : \text{Frame}, \text{Realized } F \rightarrow P$ ) :
   $\neg \forall F : \text{Frame}, P \rightarrow \text{Value } F :=$ 
  fun h => counter_fails (h counter (hw calm act_is_keyed.1))

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no instance fails, an unrealized class. -/
theorem only_refuter (F : Frame) :  $\neg \text{Realized } F \rightarrow \neg \exists d, F.\text{realize } d = \text{none} :=$ 
  fun hn hne => hn (fun d => match hc : F.realize d with
  | some n => (n, rfl)
  | none => absurd (d, hc) hne)

structure ActualClasses where
  F : Frame
  supply : Realized F

/-- THE HODGE VALUE FROM EXISTENCE, BY ONE ACT. -/
theorem hodge_from_existence (A : ActualClasses) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the value holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualClasses // A.F = F }  $\leftrightarrow$  Value F :=
  (fun (A, hA) => hA  $\triangleright$  hodge_from_existence A,
  fun h => (((F, (act_is_the_value F).mpr h), rfl)))

theorem every_class_lands (F : Frame) (d : F.D) :
  ( $\exists n, F.\text{realize } d = \text{some } n$ )  $\vee$  F.realize d = none :=
  match F.realize d with
| some n => Or.inl (n, rfl)
| none => Or.inr rfl

theorem gates_exclusive (F : Frame) (d : F.D) :
   $\neg ((\exists n, F.\text{realize } d = \text{some } n) \wedge F.\text{realize } d = \text{none}) :=$ 
  fun (d, h, h') => Option.noConfusion (h'.symm.trans h)

theorem nothing_escapes (A : ActualClasses) :  $\forall d, \exists n, A.F.\text{realize } d = \text{some } n := A.\text{supply}$ 

/-- One unrealized Hodge class refutes the act. -/
theorem counterclass_refutes (F : Frame) (d : F.D) (h : F.realize d = none) :  $\neg \text{Realized } F :=$ 
  fun hA => match hA d with
  | (_, hn) => Option.noConfusion (h.symm.trans hn)

/-! ## IV · the retired premise -/

```

```

/-- The semiregular witness frame: a realization together with a
    semiregularity flag. -/
structure Witnessed where
  F      : Frame
  semi   : F.D → Bool

/-- The witness axiom: every class realized by a semiregular
    witness. -/
def WitnessAxiom (W : Witnessed) : Prop := ∀ d, Algebraic W.F d ∧
  W.semi d = true

theorem witness_axiom_gives_value (W : Witnessed) (h : WitnessAxiom
  W) : Value W.F :=
  fun d => (h d).1

/-- A world with the value and no semiregular witness. -/
def plainWitnessed : Witnessed := (calm, fun _ => false)

/-- THE WITNESS AXIOM IS RETIRED: it implies the value and is
    strictly stronger. -/
theorem witness_axiom_strictly_stronger :
  Value plainWitnessed.F ∧ ¬ WitnessAxiom plainWitnessed :=
  (calm_value, fun h => Bool.noConfusion (h ()).2)

/-! ## V · the price -/

/-- An integral frame: each class carries an integral lift, `none`
    for a torsion class whose
    integral lift no cycle realizes, beside its rational realization. -/
structure Integral where
  F      : Frame
  integral : F.D → Option Nat

def torsionWorld : Integral := (calm, fun _ => none)

/-- Integrality does not transfer: the rational value holds and the
    integral one fails. -/
theorem integrality_does_not_transfer :
  Value torsionWorld.F ∧ ¬ ∀ d, ∃ n, torsionWorld.integral d =
  some n :=
  (calm_value, fun h => match h () with | (_, hn) =>
  Option.noConfusion hn)

/-- The decided slice: an integral class of codimension one is the
    first Chern class of a line
    bundle by the exponential field, and a rational one has such a
    multiple, so it is realized; the
    field is cited at its grade. -/
structure Slice where
  F      : Frame
  bundle  : F.D → Option Nat
  expo    : ∀ d, F.codim d = 1 → ∃ n, bundle d = some n
  chern   : ∀ d n, bundle d = some n → F.realize d = some n

/-- THE SLICE DECIDES ITS SLICE: every codimension-one class is
    algebraic. -/
theorem slice_decides (S : Slice) (d : S.F.D) (h1 : S.F.codim d = 1)
  : Algebraic S.F d :=
  match S.expo d h1 with
  | (n, hb) => (n, S.chern d n hb)

/-- The slice without the exponential field. -/
structure SliceNoExp where
  F      : Frame
  bundle  : F.D → Option Nat
  chern   : ∀ d n, bundle d = some n → F.realize d = some n

def bareSlice : SliceNoExp := ((Unit, fun _ => 1, fun _ => none),
  fun _ => none,
  fun _ _ h => Option.noConfusion h)

/-- THE EXPONENTIAL FIELD IS LOAD-BEARING: without it a
    codimension-one class goes unrealized. -/
theorem expo_is_load_bearing : bareSlice.F.codim () = 1 ∧ ¬
  Algebraic bareSlice.F () :=
  (rfl, fun (_, h) => Option.noConfusion h)

/-- The slice decides nothing beyond itself: a slice whose
    codimension-two class is unrealized. -/
def slicedCounter : Slice := (counter, fun _ => none,
  fun d h => absurd h (by decide : ¬ (2 = 1)), fun _ _ h =>
  Option.noConfusion h)

```

```

theorem slice_does_not_decide_the_row : ¬ Value slicedCounter.F :=
  counter_fails

/-- A slice at a codimension other than one, its one class
    unrealized. -/
def sliceCounterAt (c : Nat) (hc : c ≠ 1) : Slice :=
  ((Unit, fun _ => c, fun _ => none), fun _ => none, fun _ h =>
  absurd h hc, fun _ _ h => Option.noConfusion h)

/-- THE SLICE DECIDES NOTHING BEYOND ITSELF: at every codimension
    other than one a slice carries an
    unrealized class. -/
theorem slice_decides_nothing_beyond (c : Nat) (hc : c ≠ 1) :
  (sliceCounterAt c hc).F.codim () = c ∧ ¬ Algebraic
  (sliceCounterAt c hc).F () :=
  (rfl, fun (_, h) => Option.noConfusion h)

/-! ## VI · freedom -/

/-- The kinetic record, the Bool model: one reading for both values
    of the bit. -/
def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL IN THE BOOL MODEL: no reading of the kinetic
    record returns the bit. -/
theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) =
  w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord
  w == kineticRecord true)).length = 2 := by
  decide

/-- The record of a frame: its classes and their codimensions,
    without their realizations. -/
def rowRecord (F : Frame) : (D : Type) × (D → Nat) := (F.D, F.codim)

/-- THE RECORD IS SHARED: the algebraic world and the counter world
    carry one record. -/
theorem record_shared : rowRecord calm = rowRecord counter := rfl

/-- THE RECORD WALL ON FRAMES: no reading of the record of a frame
    is the value on every frame. -/
theorem row_record_wall :
  ¬ ∃ g : ((D : Type) × (D → Nat)) → Prop, ∀ F : Frame, (g
  (rowRecord F) ↔ Value F) :=
  fun (g, hg) => counter_fails ((hg counter).mp (Eq.mp (congrArg g
  record_shared) ((hg calm).mpr calm_value)))

/-- One class of codimension two with its realization given:
    `onRecord (some 0)` is the algebraic world and
    `onRecord none` the counter world, by definition. -/
def onRecord (o : Option Nat) : Frame := (Unit, fun _ => 2, fun _ =>
  o)

/-- THE FIBRE OF THE RECORD IS INFINITE, ITS BIT TWO: every
    realization shares the record, distinct cycles give
    distinct frames, every realized frame carries the value, and the
    unrealized one fails it. -/
theorem row_fibre_infinite_bit_two :
  (∀ o : Option Nat, rowRecord (onRecord o) = rowRecord calm) ∧
  (∀ m n : Nat, (onRecord (some m)).realize () = (onRecord (some
  n)).realize () → m = n) ∧
  (∀ n : Nat, Value (onRecord (some n))) ∧ ¬ Value (onRecord none)
  :=
  (fun _ => rfl, fun _ _ h => Option.some.inj h, fun n _ => (n,
  rfl), counter_fails)

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
  (List.range (n + 1)).filterMap fun b => if a * b = n then some
  (a, b) else none

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1
  != p.2) = true ∧
  (mulFibre 7).length = ([false, true].filter (fun w =>
  kineticRecord w == kineticRecord true)).length := by
  decide

/-- THE PRIME'S FIBRE IS ONE SWAP ORBIT: swapping the factors of
    each pair reverses the fibre of seven. -/
theorem prime_fibre_one_swap_orbit : (mulFibre 7).map (fun p =>
  (p.2, p.1)) = (mulFibre 7).reverse := by

```

```

decide

/-- Conjugation on Hodge types: the type (p,q) goes to the type
(q,p). -/
def conjType (t : Nat × Nat) : Nat × Nat := (t.2, t.1)

/-- CONJUGATION IS AN INVOLUTION: applied twice it returns every
type. -/
theorem conjType_involutive : ∀ t : Nat × Nat, conjType (conjType t)
= t :=
  fun {_, _} => rfl

/-- THE SEAT OF THE ROW: a Hodge type is fixed by conjugation
exactly when it is a (p,p) type. -/
theorem hodge_seat (p q : Nat) : conjType (p, q) = (p, q) ↔ q = p :=
  (fun h => (congrArg Prod.fst h : q = p),
   fun h => Eq.subst (motive := fun x => conjType (p, x) = (p, x))
   h.symm rfl)

/-! ## VII · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
  [false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r
!= (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
r3 =>
  r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
[false, true].all (fun t3 =>
  solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
  by decide

/-! ## VIII · the record and the seed -/

/-- The world algebraic in codimension below n and with a counter
class at codimension n. -/
def staged (n : Nat) : Frame :=
  (Nat, fun d => d, fun d => if d < n then some 0 else none)

/-- No finite record forces the value: every class below codimension
n is algebraic and the
value fails. -/
theorem finite_record_never_forces (n : Nat) :
  (∀ d, d < n → Algebraic (staged n) d) ∧ ¬ Value (staged n) :=
  (fun d hd => {0, by show (if d < n then some 0 else none) = some
0; rw [if_pos hd]},
   fun h => match h n with
   | (_, hk) => by
     have e : (staged n).realize n = none := by
       show (if n < n then some 0 else (none : Option Nat)) = none
       rw [if_neg (Nat.lt_irrefl n)]
     exact Option.noConfusion (e.symm.trans hk))

/-- A symmetry keeps the codimension: conjugation fixes the type of
a (p,p) class. -/
structure Symmetric {D : Type} (codim : D → Nat) where
  Grp : Type
  act : Grp → D → D
  keep : ∀ g d, codim (act g d) = codim d

theorem no_symmetry_lifts {D : Type} (codim : D → Nat) (S :
  Symmetric D codim) (r : Nat) (d e : D)
  (hd : codim d ≤ r) (he : r < codim e) : ∀ g, S.act g d ≠ e :=
  fun g h =>

```

```

  have k : codim e = codim d := h ▸ S.keep g d
  have h2 : codim e ≤ r := by rw [k]; exact hd
  Nat.lt_irrefl r (Nat.lt_of_lt_of_le he h2)

/-- NO SYMMETRY LIFTS ACROSS ANY CUT: whatever predicate on
codimensions decides a class, no symmetry carries a
decided class to an undecided one. -/
theorem no_symmetry_lifts_any_cut {D : Type} (codim : D → Nat) (S :
  Symmetric D codim) (Decided : Nat → Prop)
  (d e : D) (hd : Decided (codim d)) (he : ¬ Decided (codim e)) :
  ∀ g, S.act g d ≠ e :=
  fun g h =>
    have k : codim e = codim d := h ▸ S.keep g d
    he (Eq.mpr (congrArg Decided k) hd)

structure Ladder where
  Decided : Nat → Prop
  base : Decided 0
  mono : ∀ r s, Decided s → r ≤ s → Decided r
  δ : Nat → Nat
  step : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) :
  ∀ r, L.Decided r
  | 0 => L.base
  | r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
    (uniform_step_forces_all L hδ r))
    (Nat.add_le_add_left (hδ r) r)

/-! ## IX · the proved part: low dimension, sealed -/

/-- A variety of dimension at most three, with three cited theorems
carried as fields, each
stating its conclusion: the end classes (fundamental class and
points) are algebraic; the
Lefschetz (1,1) slice; and hard Lefschetz, codimension dim - 1
algebraic. The transport from
codimension one is cited, not modelled. -/
structure LowDim where
  F : Frame
  dim : Nat
  small : dim ≤ 3
  below : ∀ d, F.codim d ≤ dim
  ends : ∀ d, F.codim d = 0 ∨ F.codim d = dim → Algebraic F d
  lefl1 : ∀ d, F.codim d = 1 → Algebraic F d
  hardL : ∀ d, F.codim d + 1 = dim → Algebraic F d

/-- In dimension at most three every codimension is an end, one, or
one below the dimension. -/
theorem small_cases (c n : Nat) (h1 : c ≤ n) (h2 : n ≤ 3) :
  (c = 0 ∨ c = n) ∨ c = 1 ∨ c + 1 = n :=
  match Nat.eq_zero_or_pos c with
  | Or.inl h0 => Or.inl (Or.inl h0)
  | Or.inr hp => match Nat.lt_or_ge c 2 with
  | Or.inl hl => Or.inr (Or.inl (Nat.le_antisymm
    (Nat.le_of_lt_succ hl) hp))
  | Or.inr hg => match Nat.lt_or_ge c n with
  | Or.inr hge => Or.inl (Or.inr (Nat.le_antisymm h1 hge))
  | Or.inl hlt =>
    have a : c + 1 ≤ n := hlt
    have b : 3 ≤ c + 1 := Nat.succ_le_succ hg
    have n3 : n = 3 := Nat.le_antisymm h2 (Nat.le_trans b a)
    Or.inr (Or.inr (Nat.le_antisymm a (n3 ▸ b)))

/-- THE PROVED PART: in dimension at most three every Hodge class is
algebraic. -/
theorem low_dim_decided (L : LowDim) : Value L.F := fun d =>
  match small_cases (L.F.codim d) L.dim (L.below d) L.small with
  | Or.inl e => L.ends d e
  | Or.inr (Or.inl o) => L.lefl1 d o
  | Or.inr (Or.inr t) => L.hardL d t

/-- Dimension three without the hard Lefschetz field. -/
structure LowDimNoHL where
  F : Frame
  dim : Nat
  ends : ∀ d, F.codim d = 0 ∨ F.codim d = dim → Algebraic F d
  lefl1 : ∀ d, F.codim d = 1 → Algebraic F d

def bareThreefold : LowDimNoHL :=
  ((Unit, fun _ => 2, fun _ => none), 3,

```



```

fun _ h => absurd h (by decide :  $\neg ((2 : \text{Nat}) = 0 \vee (2 : \text{Nat}) = 3)$ ),
fun _ h => absurd h (by decide :  $\neg ((2 : \text{Nat}) = 1)$ ))

/-- THE HARD LEFSCHETZ FIELD IS LOAD-BEARING: without it a
codimension-two class on a threefold
goes unrealized while the ends and the (1,1) slice hold. -/
theorem hardL_is_load_bearing :
  bareThreefold.F.codim () + 1 = bareThreefold.dim  $\wedge$   $\neg$  Algebraic
  bareThreefold.F () :=
  (rfl, fun (_, h) => Option.noConfusion h)

/-- Dimension three without the field of the end classes. -/
structure LowDimNoEnds where
  F : Frame
  dim : Nat
  lef11 :  $\forall d, F.\text{codim } d = 1 \rightarrow$  Algebraic F d
  hardL :  $\forall d, F.\text{codim } d + 1 = \text{dim} \rightarrow$  Algebraic F d

def bareEnds : LowDimNoEnds :=
  ((Unit, fun _ => 0, fun _ => none), 3,
  fun _ h => absurd h (by decide :  $\neg ((0 : \text{Nat}) = 1)$ ),
  fun _ h => absurd h (by decide :  $\neg ((0 : \text{Nat}) + 1 = 3)$ ))

/-- THE FIELD OF THE END CLASSES IS LOAD-BEARING: without it a
codimension-zero class on a threefold goes
unrealized while the (1,1) slice and hard Lefschetz hold. -/
theorem ends_is_load_bearing :
  bareEnds.F.codim () = 0  $\wedge$   $\neg$  Algebraic bareEnds.F () :=
  (rfl, fun (_, h) => Option.noConfusion h)

/-- Dimension three without the (1,1) field. -/
structure LowDimNoLef where
  F : Frame
  dim : Nat
  ends :  $\forall d, F.\text{codim } d = 0 \vee F.\text{codim } d = \text{dim} \rightarrow$  Algebraic F d
  hardL :  $\forall d, F.\text{codim } d + 1 = \text{dim} \rightarrow$  Algebraic F d

def bareLef : LowDimNoLef :=
  ((Unit, fun _ => 1, fun _ => none), 3,
  fun _ h => absurd h (by decide :  $\neg ((1 : \text{Nat}) = 0 \vee (1 : \text{Nat}) = 3)$ ),
  fun _ h => absurd h (by decide :  $\neg ((1 : \text{Nat}) + 1 = 3)$ ))

/-- THE LEFSCHETZ (1,1) FIELD IS LOAD-BEARING: without it a
codimension-one class on a threefold goes
unrealized while the end classes and hard Lefschetz hold. -/
theorem lef11_is_load_bearing :
  bareLef.F.codim () = 1  $\wedge$   $\neg$  Algebraic bareLef.F () :=
  (rfl, fun (_, h) => Option.noConfusion h)

/-- The cited fields in every dimension, with no bound on the
dimension: the end classes, the (1,1) slice
and hard Lefschetz. -/
structure Outer where
  F : Frame
  dim : Nat
  below :  $\forall d, F.\text{codim } d \leq \text{dim}$ 
  ends :  $\forall d, F.\text{codim } d = 0 \vee F.\text{codim } d = \text{dim} \rightarrow$  Algebraic F d
  lef11 :  $\forall d, F.\text{codim } d = 1 \rightarrow$  Algebraic F d
  hardL :  $\forall d, F.\text{codim } d + 1 = \text{dim} \rightarrow$  Algebraic F d

/-- THE OUTER CODIMENSIONS ARE DECIDED IN EVERY DIMENSION: every
class outside the middle range
 $2 \leq p \leq n - 2$ , the ends, codimension one and codimension  $n - 1$ , is
realized by the cited fields. -/
theorem outer_decided (O : Outer) (d : O.F.D)
  (h :  $\neg (2 \leq O.F.\text{codim } d \wedge O.F.\text{codim } d + 2 \leq 0.\text{dim})$ ) : Algebraic
  O.F d :=
  match Nat.eq_zero_or_pos (O.F.codim d) with
  | Or.inl h0 => 0.ends d (Or.inl h0)
  | Or.inr hp => match Nat.lt_or_ge (O.F.codim d) 2 with
  | Or.inl h1 => 0.lef11 d (Nat.le_antisymm (Nat.le_of_lt_succ h1) hp)
  | Or.inr hg =>
    have hn : 0.dim  $\leq$  O.F.codim d + 1 :=
      Nat.le_of_lt_succ (Nat.lt_of_not_le (fun hle => h (hg, hle)))
    match Nat.lt_or_ge (O.F.codim d) 0.dim with
    | Or.inr hge => 0.ends d (Or.inr (Nat.le_antisymm (0.below d) hge))
    | Or.inl hlt => 0.hardL d (Nat.le_antisymm hlt hn)

/-- A fourfold carrying every cited field, with one class of
codimension two, in the middle range, unrealized. -/

```

```

def bareFourfold : Outer :=
  ((Unit, fun _ => 2, fun _ => none), 4, fun _ => (by decide : (2 :
  Nat)  $\leq$  4),
  fun _ h => absurd h (by decide :  $\neg ((2 : \text{Nat}) = 0 \vee (2 : \text{Nat}) = 4)$ ),
  fun _ h => absurd h (by decide :  $\neg ((2 : \text{Nat}) = 1)$ ),
  fun _ h => absurd h (by decide :  $\neg ((2 : \text{Nat}) + 1 = 4)$ ))

/-- THE MIDDLE IS NOT FORCED: from dimension four the cited fields,
which realize every outer codimension, leave a
class of the middle range unrealized; the remainder of the proved
part is exactly the middle codimension. -/
theorem middle_not_forced :
  (2  $\leq$  bareFourfold.F.codim ()  $\wedge$  bareFourfold.F.codim () + 2  $\leq$ 
  bareFourfold.dim)  $\wedge$ 
   $\neg$  Algebraic bareFourfold.F () :=
  ((by decide : (2 : Nat)  $\leq$  2), (by decide : (2 : Nat) + 2  $\leq$  4)),
  fun (_, h) => Option.noConfusion h)

/-- The middle act: every class of the middle range  $2 \leq p \leq n - 2$  is
realized. -/
def MiddleRealized (O : Outer) : Prop :=
   $\forall d, 2 \leq O.F.\text{codim } d \wedge O.F.\text{codim } d + 2 \leq 0.\text{dim} \rightarrow$  Algebraic O.F d

/-- THE MIDDLE ACT CLOSES: on every frame carrying the cited fields,
the middle act is exactly the value. -/
theorem middle_act_closes (O : Outer) : MiddleRealized O  $\leftrightarrow$  Value O.F
:=
  (fun h d => match Nat.decLe 2 (O.F.codim d), Nat.decLe (O.F.codim
  d + 2) 0.dim with
  | isTrue h1, isTrue h2 => h d (h1, h2)
  | isFalse h1, _ => outer_decided O d (fun hm => h1 hm.1)
  | isTrue _, isFalse h2 => outer_decided O d (fun hm => h2 hm.2),
  fun h d _ => h d)

/-- An n-fold carrying every cited field, with one class of
codimension p, in the middle range, unrealized. -/
def middleWitness (n p : Nat) (h2 : 2  $\leq$  p) (hn : p + 2  $\leq$  n) : Outer :=
  ((Unit, fun _ => p, fun _ => none), n, fun _ => Nat.le_trans
  (Nat.le_add_right p 2) hn,
  fun _ h => h.elim
  (fun h0 => absurd (Nat.le_trans h2 (Nat.le_of_eq h0)) (by
  decide :  $\neg (2 : \text{Nat}) \leq 0$ ))
  (fun hpn => absurd (Nat.le_trans (Nat.le_add_right (p + 1) 1)
  (Nat.le_trans hn (Nat.le_of_eq hpn.symm)))
  (Nat.not_succ_le_self p)),
  fun _ h => absurd (Nat.le_trans h2 (Nat.le_of_eq h)) (by decide :
   $\neg (2 : \text{Nat}) \leq 1$ ),
  fun _ h => absurd (Nat.le_trans hn (Nat.le_of_eq h.symm))
  (Nat.not_succ_le_self (p + 1)))

/-- EVERY MIDDLE CODIMENSION IS UNFORCED: in every dimension, at
every codimension of the middle range, the cited
fields leave a class unrealized. -/
theorem every_middle_codimension_unforced (n p : Nat) (h2 : 2  $\leq$  p)
  (hn : p + 2  $\leq$  n) :
  (middleWitness n p h2 hn).dim = n  $\wedge$  (middleWitness n p h2
  hn).F.codim () = p  $\wedge$ 
   $\neg$  Algebraic (middleWitness n p h2 hn).F () :=
  (rfl, rfl, fun (_, h) => Option.noConfusion h)

/-- Decided below r: every class of codimension below r is realized.
-/
def decidedBelow (O : Outer) (r : Nat) : Prop :=  $\forall d, O.F.\text{codim } d <
r \rightarrow$  Algebraic O.F d

/-- THE CITED FIELDS CARRY NO UNIFORM STEP: on the bare fourfold,
decided below r admits no step of at least one at
every rung, since such a step would decide every rung and realize
the middle class. -/
theorem cited_fields_carry_no_step :
   $\neg \exists \delta : \text{Nat}, (\forall r, 1 \leq \delta r) \wedge \forall r, \text{decidedBelow}
  \text{bareFourfold } r \rightarrow \text{decidedBelow } \text{bareFourfold } (r + \delta r) :=
  fun (\delta, h\delta, hs) =>
    middle_not_forced.2
    (uniform_step_forces_all (decidedBelow bareFourfold, fun _ h
    => absurd h (Nat.not_lt_zero _),
    fun _ _ hs' hrs d hd => hs' d (Nat.lt_of_lt_of_le hd hrs),
    \delta, hs) h\delta 3 ()) (by decide : (2 : Nat) < 3))

/-- ## X · the division: the proved part and its remainder -/

/-- The value on the classes a cut selects. -/
def ValueOn (F : Frame) (cut : F.D  $\rightarrow$  Bool) (b : Bool) : Prop :=$ 
```

```

    ∀ d, cut d = b → Algebraic F d

/-- THE CUT-AGNOSTIC DIVISION: for every cut, the value is exactly
its two halves. -/
theorem row_split (F : Frame) (cut : F.D → Bool) :
  Value F ↔ ValueOn F cut true ∧ ValueOn F cut false :=
  (fun h => (fun d _ => h d, fun d _ => h d),
   fun (ht, hf) d => match hc : cut d with
   | true => ht d hc
   | false => hf d hc)

/-- The decided half is proved on every slice frame: the
codimension-one half holds. -/
theorem proved_half (S : Slice) :
  ValueOn S.F (fun d => Nat.beq (S.F.codim d) 1) true :=
  fun d h => slice_decides S d (Nat.eq_of_beq_eq_true h)

/-- The remainder is not forced by the proved half: in the counter
world the codimension-one half
holds vacuously and the remainder fails. -/
theorem remainder_not_forced :
  ValueOn counter (fun d => Nat.beq (counter.codim d) 1) true ∧
  ¬ ValueOn counter (fun d => Nat.beq (counter.codim d) 1) false :=
  (fun _ h => absurd h (by decide : ¬ (Nat.beq 2 1 = true))),
  fun h => counter_fails (fun d => h d rfl))

/-! ## XI · least escape: keyed, never keyless -/

/-- No keyless statement, reading the same on every frame, is the
act. -/
theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ F : Frame,
  (P ↔ Realized F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans
  (act_is_the_value F))

/-- The pulse does not certify: the root holds vacuously on the
empty background beside a world whose
value fails. -/
theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧
  Value counter :=
  (root_given, counter_fails)

/-! ## XII · the closure, whole -/

/-- THE HODGE CLOSURE: existence as given forces no value and holds
in both worlds; the act is the
value and closes it; one counter class refutes it; the witness axiom
is strictly stronger; the
proved part decides dimension at most three and the outer
codimensions in every dimension; the middle
codimension is unforced from dimension four, at every middle
codimension of every dimension; and the
proved half holds where the remainder fails. -/
theorem hodge_closure :
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
  ((Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value counter)) ∧
  (∀ F : Frame, Realized F ↔ Value F) ∧
  (∀ A : ActualClasses, Value A.F) ∧
  (∀ (F : Frame) (d : F.D), F.realize d = none → ¬ Realized F) ∧
  (Value plainWitnessed.F ∧ ¬ WitnessAxiom plainWitnessed) ∧
  (∀ L : LowDim, Value L.F) ∧
  (∀ (0 : Outer) (d : 0.F.D), ¬ (2 ≤ 0.F.codim d ∧ 0.F.codim d + 2
  ≤ 0.dim) → Algebraic 0.F d) ∧
  ((2 ≤ bareFourfold.F.codim () ∧ bareFourfold.F.codim () + 2 ≤
  bareFourfold.dim) ∧
  ¬ Algebraic bareFourfold.F ()) ∧
  (∀ (n p : Nat) (h2 : 2 ≤ p) (hn : p + 2 ≤ n), (middleWitness n p
  h2 hn).dim = n ∧
  (middleWitness n p h2 hn).F.codim () = p ∧ ¬ Algebraic
  (middleWitness n p h2 hn).F ()) ∧
  (ValueOn counter (fun d => Nat.beq (counter.codim d) 1) true ∧
  ¬ ValueOn counter (fun d => Nat.beq (counter.codim d) 1)
  false) :=
  (given_is_not_the_value, given_in_both_worlds, act_is_the_value,
  hodge_from_existence,
  counterclass_refutes, witness_axiom_strictly_stronger,
  low_dim_decided, outer_decided,
  middle_not_forced, every_middle_codimension_unforced,
  remainder_not_forced)

/-! ## The root, undeniable in deed -/

```

```

/-- A self-grounding root, the structure of Appendix R in the same
words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- SELF-GROUNDING IS EXACTLY HOLDING: a proposition has a
self-grounding exactly when it holds. -/
theorem self_grounding_iff (R : Prop) : Nonempty (SelfGrounding R) ↔
  R :=
  (fun (G) => G.instances G.anAct, fun r => ((Unit, ()), fun _ => r))

/-- The constructed root, self-grounding: one act, which instances
it. -/
def constructedGround : SelfGrounding (Root Unit (fun _ => 1)) :=
  (Unit, ()), fun _ => constructed_root_holds

/-- THE ROW'S ROOT IS GROUNDED EXACTLY WHERE THE VALUE HOLDS: the
root read on a frame has a self-grounding
exactly when the frame carries the value. -/
theorem row_root_grounds_iff_value (F : Frame) : Nonempty
  (SelfGrounding (RootOnRow F)) ↔ Value F :=
  (self_grounding_iff (RootOnRow F)).trans (root_on_row_is_the_value
  F)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
in a frame where the value holds
and in one where it fails, so it neither forces the value on every
frame nor refutes it on every frame. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R ∧ Value calm ∧ ¬ Value counter ∧ ¬ (∀ F : Frame, R → Value F) ∧
  ¬ (∀ F : Frame, R → ¬ Value F) :=
  (G.instances G.anAct, calm_value, counter_fails, fun h =>
  counter_fails (h counter (G.instances G.anAct)),
  fun h => h calm (G.instances G.anAct) calm_value)

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
uniformly on the frames it gives no value;
read on the row, with the row's existents and the row's actuation,
it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R ∧ ¬ (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F ↔
  Value F) ∧ (RootOnRow calm ∧ ¬ RootOnRow counter) :=
  (G.instances G.anAct, fun h => counter_fails (h counter
  (G.instances G.anAct)), root_on_row_is_the_value,
  root_on_row_is_keyed)

/-- The actuation of the root of Appendix R: one at its one
existent. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-- The root of Appendix R, in its own words: every existent
actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The root of Appendix R, self-grounding: one act, which instances
it. -/
def ra₀ : SelfGrounding RA₀ := (Unit, ()), fun _ => show (0 : Int)
  < 1 by decide)

/-- THE ROOT OF APPENDIX R IS THE CONSTRUCTED ROOT: the two read
alike, existent by existent. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1)
  := Iff.rfl

/-- THE LOCK ON THE ROOT: the root of Appendix R holds; read
uniformly on the frames it gives no value; read on
the row it is the value exactly; its row reading is grounded in the
algebraic world and not in the counter

```

```

world. -/
theorem the_lock_on_the_root :
  RA0  $\wedge$   $\neg$  ( $\forall F : \text{Frame}, RA_0 \rightarrow \text{Value } F$ )  $\wedge$  ( $\forall F : \text{Frame}, \text{RootOnRow } F \leftrightarrow \text{Value } F$ )  $\wedge$ 
  Nonempty (SelfGrounding (RootOnRow calm))  $\wedge$   $\neg$  Nonempty
  (SelfGrounding (RootOnRow counter)) :=
  (ra0.instances ra0.anAct, fun h => counter_fails (h counter
  (ra0.instances ra0.anAct))),
  root_on_row_is_the_value, (row_root_grounds_iff_value calm).mpr
  calm_value,
  fun h => counter_fails ((row_root_grounds_iff_value counter).mp
  h))

/-- THE LOCK, one theorem on no axiom: the constructed root holds
  with no hypothesis; every self-grounding
  root holds by its deed; no given statement is the value on every
  frame; the root read uniformly on the frames
  gives no value; read on the row the root is the act, the act is the
  value exactly, and both are keyed; the
  closing theorem gives the value on every actual frame; no reading of
  the kinetic record returns the bit; and
  no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1)  $\wedge$ 
  R  $\wedge$ 
  ( $\forall P : \text{Prop}, \neg \forall F : \text{Frame}, (P \leftrightarrow \text{Value } F)$ )  $\wedge$ 
   $\neg$  ( $\forall F : \text{Frame}, R \rightarrow \text{Value } F$ )  $\wedge$ 
  ( $\forall F : \text{Frame}, \text{RootOnRow } F \leftrightarrow \text{Realized } F$ )  $\wedge$ 
  ( $\forall F : \text{Frame}, \text{Realized } F \leftrightarrow \text{Value } F$ )  $\wedge$ 
  (RootOnRow calm  $\wedge$   $\neg$  RootOnRow counter)  $\wedge$ 
  (Realized calm  $\wedge$   $\neg$  Realized counter)  $\wedge$ 
  ( $\forall A : \text{ActualClasses}, \text{Value } A.F$ )  $\wedge$ 
  ( $\neg \exists g : \text{Nat} \rightarrow \text{Bool}, \forall w, g$  (kineticRecord w) = w)  $\wedge$ 
  ( $\forall n : \text{Nat}, (\forall d, d < n \rightarrow \text{Algebraic (staged } n) d}) \wedge \neg \text{Value}$ 
  (staged n)) :=
  (constructed_root_holds, G.instances G.anAct,
  given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
  root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
  act_is_keyed, hodge_from_existence, record_wall,
  finite_record_never_forces)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the
  closing theorem consumes existence read on
  the row and nothing else; that reading is available on exactly the
  frames where the value holds; and no
  statement given without the reading is the value on every frame. One
  theorem, on no axiom. -/
theorem from_existence_alone :
  ( $\forall A : \text{ActualClasses}, \text{Value } A.F$ )  $\wedge$ 
  ( $\forall F : \text{Frame}, \text{Nonempty } \{ A : \text{ActualClasses} // A.F = F \} \leftrightarrow \text{Value}$ 
  F)  $\wedge$ 
  ( $\forall P : \text{Prop}, \neg \forall F : \text{Frame}, (P \leftrightarrow \text{Value } F)$ ) :=
  (hodge_from_existence, supply_iff, given_is_not_the_value)

end HodgeClose

/-! ## Cones, pinned as printed -/
/-- info: 'HodgeClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.calm_value
/-- info: 'HodgeClose.counter_fails' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.counter_fails
/-- info: 'HodgeClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.root_given
/-- info: 'HodgeClose.root_conservative' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.root_conservative
/-- info: 'HodgeClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.arrow_given
/-- info: 'HodgeClose.freedom_given' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.freedom_given
/-- info: 'HodgeClose.given_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.given_is_not_the_value
/-- info: 'HodgeClose.arrow_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.arrow_forces_nothing
/-- info: 'HodgeClose.given_in_both_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.given_in_both_worlds
/-- info: 'HodgeClose.act_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.act_is_the_value

/-- info: 'HodgeClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.act_is_keyed
/-- info: 'HodgeClose.hodge_from_existence' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.hodge_from_existence
/-- info: 'HodgeClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.supply_iff
/-- info: 'HodgeClose.every_class_lands' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.every_class_lands
/-- info: 'HodgeClose.gates_exclusive' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.gates_exclusive
/-- info: 'HodgeClose.nothing_escapes' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.nothing_escapes
/-- info: 'HodgeClose.counterclass_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.counterclass_refutes
/-- info: 'HodgeClose.witness_axiom_gives_value' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.witness_axiom_gives_value
/-- info: 'HodgeClose.witness_axiom_strictly_stronger' does not
depend on any axioms -/
#guard_msgs in #print axioms
  HodgeClose.witness_axiom_strictly_stronger
/-- info: 'HodgeClose.integrality_does_not_transfer' does not depend
on any axioms -/
#guard_msgs in #print axioms
  HodgeClose.integrality_does_not_transfer
/-- info: 'HodgeClose.slice_decides' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.slice_decides
/-- info: 'HodgeClose.expo_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.expo_is_load_bearing
/-- info: 'HodgeClose.slice_does_not_decide_the_row' does not depend
on any axioms -/
#guard_msgs in #print axioms
  HodgeClose.slice_does_not_decide_the_row
/-- info: 'HodgeClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.record_wall
/-- info: 'HodgeClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.fibre_is_two
/-- info: 'HodgeClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.prime_shape
/-- info: 'HodgeClose.two_axes_leave_two' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.two_axes_leave_two
/-- info: 'HodgeClose.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.three_axes_lock_one
/-- info: 'HodgeClose.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.finite_record_never_forces
/-- info: 'HodgeClose.no_symmetry_lifts' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.no_symmetry_lifts
/-- info: 'HodgeClose.uniform_step_forces_all' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.uniform_step_forces_all
/-- info: 'HodgeClose.small_cases' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.small_cases
/-- info: 'HodgeClose.low_dim_decided' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.low_dim_decided
/-- info: 'HodgeClose.hardL_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.hardL_is_load_bearing
/-- info: 'HodgeClose.row_split' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.row_split
/-- info: 'HodgeClose.proved_half' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.proved_half
/-- info: 'HodgeClose.remainder_not_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.remainder_not_forced
/-- info: 'HodgeClose.no_keyless_statement_is_the_act' does not
depend on any axioms -/
#guard_msgs in #print axioms
  HodgeClose.no_keyless_statement_is_the_act
/-- info: 'HodgeClose.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.pulse_does_not_certify

```

```

/-- info: 'HodgeClose.hodge_closure' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.hodge_closure
/-- info: 'HodgeClose.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.denial_reenacts_root
/-- info: 'HodgeClose.external_proof_adds_nothing' does not depend
on any axioms -/
#guard_msgs in #print axioms HodgeClose.external_proof_adds_nothing
/-- info: 'HodgeClose.undeniable_root_forces_no_value' does not
depend on any axioms -/
#guard_msgs in #print axioms
HodgeClose.undeniable_root_forces_no_value
/-- info: 'HodgeClose.root_read_on_row_is_keyed' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.root_read_on_row_is_keyed
/-- info: 'HodgeClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.the_lock
/-- info: 'HodgeClose.from_existence_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.from_existence_alone
/-- info: 'HodgeClose.indicator_pos' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.indicator_pos
/-- info: 'HodgeClose.root_satisfiable' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.root_satisfiable
/-- info: 'HodgeClose.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.constructed_root_holds
/-- info: 'HodgeClose.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.root_on_row_is_the_act
/-- info: 'HodgeClose.root_on_row_is_the_value' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.root_on_row_is_the_value
/-- info: 'HodgeClose.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.root_on_row_is_keyed
/-- info: 'HodgeClose.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.act_is_the_weakest_premise
/-- info: 'HodgeClose.existence_as_given_in_both_worlds' does not
depend on any axioms -/
#guard_msgs in #print axioms
HodgeClose.existence_as_given_in_both_worlds
/-- info: 'HodgeClose.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.nothing_given_refutes
/-- info: 'HodgeClose.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.only_refuter
/-- info: 'HodgeClose.ends_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.ends_is_load_bearing
/-- info: 'HodgeClose.lef11_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.lef11_is_load_bearing
/-- info: 'HodgeClose.outer_decided' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.outer_decided
/-- info: 'HodgeClose.middle_not_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.middle_not_forced
/-- info: 'HodgeClose.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.nothing_given_forces
/-- info: 'HodgeClose.no_weaker_given_premise_closes' does not
depend on any axioms -/
#guard_msgs in #print axioms
HodgeClose.no_weaker_given_premise_closes
/-- info: 'HodgeClose.slice_decides_nothing_beyond' does not depend
on any axioms -/
#guard_msgs in #print axioms HodgeClose.slice_decides_nothing_beyond
/-- info: 'HodgeClose.record_shared' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.record_shared
/-- info: 'HodgeClose.row_record_wall' does not depend on any axioms
-/
#guard_msgs in #print axioms HodgeClose.row_record_wall
/-- info: 'HodgeClose.row_fibre_infinite_bit_two' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.row_fibre_infinite_bit_two
/-- info: 'HodgeClose.prime_fibre_one_swap_orbit' does not depend on
any axioms -/

```

```

#guard_msgs in #print axioms HodgeClose.prime_fibre_one_swap_orbit
/-- info: 'HodgeClose.conjType_involutive' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.conjType_involutive
/-- info: 'HodgeClose.hodge_seat' does not depend on any axioms -/
#guard_msgs in #print axioms HodgeClose.hodge_seat
/-- info: 'HodgeClose.no_symmetry_lifts_any_cut' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.no_symmetry_lifts_any_cut
/-- info: 'HodgeClose.middle_act_closes' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.middle_act_closes
/-- info: 'HodgeClose.every_middle_codimension_unforced' does not
depend on any axioms -/
#guard_msgs in #print axioms
HodgeClose.every_middle_codimension_unforced
/-- info: 'HodgeClose.cited_fields_carry_no_step' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.cited_fields_carry_no_step
/-- info: 'HodgeClose.self_grounding_iff' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.self_grounding_iff
/-- info: 'HodgeClose.row_root_grounds_iff_value' does not depend on
any axioms -/
#guard_msgs in #print axioms HodgeClose.row_root_grounds_iff_value
/-- info: 'HodgeClose.RA0_is_the_constructed_root' does not depend
on any axioms -/
#guard_msgs in #print axioms HodgeClose.RA0_is_the_constructed_root
/-- info: 'HodgeClose.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms HodgeClose.the_lock_on_the_root

```

APPENDIX R · THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, T0E_Zero.lean, is carried verbatim from the root paper (Islam 2026h). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x) (RA_0)$. On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to

sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the_master_seal); it states the three counts side by side and no map between them. The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```

/-
  TOE_Zero.lean · Theory of Theories of Everything (TOE of All
  TOEs): Existence Proves Existence Only by Motion
  The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
  axiom declared, no sorry, no native_decide, and
  every theorem rests on no axiom at all, not even propositional
  extensionality, quotients or choice.
  I · The self-grounding root. Acts occur and every act instances
  the root. Then every denial is an act and
    re-enacts the root, the root is held by the act itself, and no
    external proof adds anything to it: the
      Omega Boundary, proved for every root that grounds itself,
      universally, by quantification.
  II · The constructed root. On the constructed domain the Root
  Axiom is a theorem.
  III · The Return and the line. On the integer quaternions i·j·k =
  -1 and each unit squares to -1; conjugation is
    an involution and its fixed set on the chart is exactly the
    scalar line.
  IV · The twelve gates, recovered three ways inside the integers of
  the quaternions: the twenty-four Hurwitz units
    counted twelve up to sign; the twelve even permutations with
    class equation 1, 3, 4, 4; and the norm-two
    shell of twenty-four, split twelve and twelve.
  V · The regress terminated. Above a self-grounding root no level
  grounds it further.
  VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 .. 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
  => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involution_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all
odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
  0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates -2 .. 2 hold every unit (doubled norm four)
and every element of the norm-two shell (doubled
norm eight), since a doubled coordinate of absolute value three
or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
  => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p

```

```

if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
  0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
    8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
  twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
    2, 3].flatMap fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
      == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
    j)).countP
  fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
| [] => [n]
| m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
  class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
| 0, _ => []
| _ + 1, [] => []
| fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
  further ground of the root is idle; the root needs
  no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
  on the constructed domain; the Return closes;
  the line is the fixed set; and the twelve gates are recovered
  three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
    : Prop, (Q → R) → R)) ∧
  RA₀ ∧ (qi.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧

```

```

(units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
(shell.filter upToSign).length = 12 ∧
collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
= [1, 3, 4, 4] :=
(the_floor_is_universal, root_on_the_constructed_domain,
the_return, the_line_is_the_fixed_set,
twenty_four_units_twelve_gates.2, the_class_equation.1,
the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
  on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, *Triaxial Actuation.lean* (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation_is_triaxial, registration_is_forced, the_only_special_cut, tongue_freedom, every_root_actuates, the_root_carries_three_axes). The root's grade is read in the third kernel, R.3, *Root Grade Ledger.lean* (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026k), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its

statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026j), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
Triaxial_Actuation.lean · actuation is triaxial

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

An actuation is a self-grounding root whose acts carry a
direction: each act has a side and a height,
and reversing the deed reverses the side and keeps the height.
From the direction alone the kernel
constructs three axes and proves them forced:
I The fold: reversing the deed is an involution.
II The registration: it keeps the height and forgets the side;
it is blind to the fold, closed with
it under combination, and it is the only height-keeping map
onto the seat.
III The seat: the fold's fixed set, where the fold and the
registration both fix the act, and off
which neither does.
IV The third axis: off the seat the fold's orbit has two acts
with one record, so the record leaves
exactly the orientation open; every reading blind to the fold
reads both alike.
V The root through the axes: a reversed deed is still an act
and instances the root; the axes are
carried by every actuation, and they force no orientation.
VI The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct     : Act

```

```

instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir       : Direction
  Height    : Type
  anAct     : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
  fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
  cases p with
  | mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
  show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height) :
reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
  (keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
  intro p
  have hs : (g p).1 = A.dir.zero := by
    have hf := lands p
    have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
    exact A.dir.fix_zero _ h1

```

```

have hh := keeps p
cases hg : g p with
| mk s h =>
  rw [hg] at hs hh
  show (s, h) = (A.dir.zero, p.2)
  rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
  (fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
  fun h => by
    cases p with
    | mk s t =>
      show (A.dir.rev s, t) = (s, t)
      have hs : s = A.dir.zero := h
      rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
  registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : OnSeat A p) :
  fold A p = p ∧ reg A p = p := by
  refine (h, ?_)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
  fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
  (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
  have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
  exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
  and its reversal are two acts with
  one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
  A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
  alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
  A.dir.Side × A.Height → α)
  (blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
  f (fold A p) = f p :=
  blind p

/-- The registration is such a reading, and so is every reading of
  the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
  Type} (k : A.dir.Side × A.Height → α)
  (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
  := rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
  denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
  A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
  are acts, both instance the root, and
  the record reads them alike; the triaxial structure leaves the one
  orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
  × A.Height) (h : ¬ OnSeat A p) :

```

```

  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, ¬(-s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc n => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc n, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
  self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↔ (-d, t), the
  registration (d, t) ↔ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
  fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
  when d = 0, the registration
  leaves it unchanged exactly then, and off the seat the record leaves
  the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧
  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
    = reg atom (d, t)) :=
  (seat_iff_zero_side atom (d, t),
  (fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
  fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
    ((seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
  one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
    : Int)) :=
  (fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
  involution, a registration that
  keeps the height, forgets the side and is the only such map onto the
  seat, and a seat where both fix
  the act and off which neither does; off the seat the record leaves
  exactly the orientation; every act
  and its reversal instance the root; and the axes force no
  orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
    (reg A p) = reg A p) ∧
  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
    (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg
      A p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
  (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
    reg A p) :=
  (fold_involutive A, fun p => (reg_forgets_side A p,
    reg_keeps_height A p, fold_of_reg A p),
  registration_is_forced A, seat_cancels_both A, off_seat_both A,
  axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
  bridge atom is an actuation; and it is
  not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
      p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
    atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
      : Int))) :=

```

```

(fun _ A => (fold_involutive A, seat_cancels_both A,
  fun p h => record_leaves_the_orientation A p h),
  fun d t => seat_iff_zero_side atom (d, t),
  the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (p y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :
  (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
reversal alike and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
  A.dir.Side × A.Height) :
  reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  (rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
  Prod.snd h, fun h => by
  cases p with
  | mk s t =>
  cases q with
  | mk s' t' =>
  have ht : t = t' := h
  show (A.dir.zero, t) = (A.dir.zero, t')
  rw [ht]))

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A (reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
its reversal land on one seat point, the record
of both; at that place the record has left exactly the orientation
open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) :
  OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
fold, the record among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) {α : Type}
  (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
  q) :
  fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
  A.dir.Side × A.Height)
  (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>

```

```

cases q with
| mk s' t' =>
  have ht : t = t' := congrArg Prod.snd hr
  have hs' : s = s' := hs
  rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
on the seat, and the fold and the
registration are both the identity: one scalar, no fold, no seat to
lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
  A.dir.Degenerate) (p : A.dir.Side × A.Height) :
  OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
actuation over its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the
root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
  (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
  fun a h =>
  have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
  ((1 : Int), a)).mp h
  absurd h0 (by decide),
  intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
actuation carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only
height-keeping map onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height
an act off the seat, its reversal a second
act with the same record, both instanting the root, the orientation
between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
  S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
  OnSeat S.toActuation (g p)) →
  ∀ p, g p = reg S.toActuation p) ∧
  (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
  S.toActuation p = p) ∧
  (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
  reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
  S.toActuation ((1 : Int), a)) ∧
  (∀ p, ¬ OnSeat S.toActuation p →
  R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
  S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
  fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
  S.toActuation p),
  registration_is_forced S.toActuation,
  seat_cancels_both S.toActuation,
  fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
  a) ((every_root_actuates S).2.1 a),
  axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/

```



```

#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no

```

axiom at all, not even propositional extensionality, quotients or choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every claim on one ledger and joins claims at the weakest link. It declares the root as `axiom RA` and pins `raClaim.grade = .premise`; its co-location claim, the root joined with the wall, therefore inherits `.premise`. This kernel carries the same ledger and the same join law with no axiom declared, warrants the root by a theorem with no axiom and no hypothesis, and proves that the co-location claim then stands at theorem grade.

I The grade ledger and the weakest-link join, as in the Codex.
II The root, a theorem: the constructed domain of TOE_Zero.lean, warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a function of its record, proved on no axiom through Tongue freedom.
IV The claims: the root at theorem grade, the wall at theorem grade, and their co-location.
V Why the Codex read premise: a join's grade is its weakest link, and co-location's grade is the root's grade exactly; with the root at theorem grade, co-location is a theorem.
VI The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a root with a warrant, a proof of its statement, is theorem grade; a root posited without one, the Codex's `axiom RA`, is premise grade; the two claims of section IV carry exactly the grades this rule reads.

-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

```

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

```

```

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
|.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
|.analytic => 7 | .theorem => 8

```

```

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

```

/- A claim carries its statement, its grade and its warrant, a proof of the statement. -/

```

structure Claim : Type where
stmt      : Prop
grade     : Grade
warrant   : stmt

```

```

/- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
stmt      := c₁.stmt ∧ c₂.stmt
grade     := Grade.weakest c₁.grade c₂.grade
warrant   := (c₁.warrant, c₂.warrant)

```

```

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
Grade.weakest c₁.grade c₂.grade := rfl

```

/-! ## II · the root, a theorem (the constructed domain of TOE_Zero.lean) -/

```

structure SelfGrounding (R : Prop) where
Act      : Type
anAct    : Act
instances : Act → R

```

```

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

```

```

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ => show (0 : Int)
< 1 by decide)

```

```

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA0 := seated_undeniable
ra0

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (p y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i
  % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero
  target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim0 : Claim where
  stmt := RA0
  grade := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
Codex. -/
def colocClaim0 : Claim := Claim.join raClaim0 wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
the same statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt := RA0
  grade := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
grade. -/

```

```

theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
= g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
theorem, so the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
  raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim0.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim0.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
both proved. -/
theorem coloc_holds : colocClaim0.stmt := colocClaim0.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
hypothesis; the wall is a theorem; their
co-location holds and stands at theorem grade; and co-location's
grade is the root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA0 ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim0.stmt ∧ colocClaim0.grade = .theorem ∧
  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_is_theorem_grade,
  coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it
-/

/-- The evidence a ledger entry carries for a statement: posited
with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
is a proof with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem
grade; a root posited with none, as the Codex
declares "axiom RA", is premise grade. -/
def rootGrade : Evidence RA0 → Grade
| .warranted _ => .theorem
| .posited => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
:= rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
reads premise, the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
.premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
carries the grade the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none.
-/
theorem the_ledger_reads_the_warrant :
  raClaim0.grade = rootGrade (.warranted raClaim0.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
and premise from none, never the other way;

```

```

whatever label an entry carries, the warranted root's grade under
the rule is theorem. -/
theorem the_grade_is_not_elected :
  (V w : RA0, rootGrade (.warranted w) = .theorem) ∧ rootGrade
  .posited ≠ .theorem ∧
  (V w : RA0, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
    Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
  RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in Islam (2026i).

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the paper does not state that the Hodge value follows from

existence as given, because the kernel's own given_is_not_the_value proves the contrary.

The register. In the programme's vocabulary: the form, the act's identity with the value, the universal closure and the retirement of the witness axiom stand at $[\{\epsilon\}, T]$ on no axiom; the proved part at the grade of its cited fields, its covering and its load-bearing checks on no axiom; the triaxial reading at $[\{\epsilon\}, S]$; the value in middle codimension from dimension four at premise grade on the act; $\Delta M = 0$ on the cited geometry. The act is the row's least-erasure posit read as existence realized; the row carries no defeater. The two-world division, prime-as-freedom and the triaxial lock are applied on the row, with the lessons of the Riemann, P versus NP and Navier--Stokes closures combined: the division of the Riemann closure, the keyed least escape of the P versus NP work, and the load-bearing check of the Navier--Stokes closure. The semiregular witness axiom of the earlier paper is retired as a placeholder.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not char-

acterless, while a degenerate direction over the same deeds would be (every_root_actuates, degenerate_is_characterless). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat the registration is the only height-keeping map onto the seat the axes force no orientation	every_root_actuates, the_root_carries_three_axes, actuation_is_triaxial, registration_is_forced (Appendix R.2) registration_is_forced (R.2) axes_force_no_orientation (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	fold, denial_instances_root (R.2); denial_reenacts_root, denial_instantiates (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	registration_is_the_folds_record, reg_forgets_side, reg_keeps_height, fold_involutive, fold_of_reg (R.2) readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026k, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)

Clause of the statement	Receipt
the seat is the fixed set, where both blind spots cancel; off the seat both stand	seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)
the bridge atom is an actuation, and not characterless	the_bridge_atom, the_atom_is_not_characterless, the_triaxial_seal (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{IF}_2^3$, two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement
the root alone forces no value; the root read on the row is the value	undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root ra_0 of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; the row's own seat is the diagonal, fixed under conjugation of Hodge types (conjType_involutive, hodge_seat), a statement of this kernel beside the actuation's seat; the row ledger of Islam (2026g, Book III), cited

Clause of the statement	Receipt

Voisin, C. 2002. A counterexample to the Hodge conjecture extended to Kähler varieties. *International Mathematics Research Notices* 2002 (20): 1057--1075.

End of manuscript

REFERENCES

- Atiyah, M. F. and F. Hirzebruch. 1962. Analytic cycles on complex manifolds. *Topology* 1: 25--45.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.
- Deligne, P. 2006. The Hodge conjecture. In *The Millennium Prize Problems*, 45--53. Clay Mathematics Institute.
- Hodge, W. V. D. 1950. The topological invariants of algebraic varieties. *Proceedings of the International Congress of Mathematicians*, Cambridge, MA, 1: 182--192.
- Islam, M. F. 2026a. A proof of the Hodge conjecture derived from one semiregular witness axiom. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026b. A formal proof of Hodge conjecture termination at the formal-alone register. Zenodo. doi:10.5281/zenodo.22705460.
- Islam, M. F. 2026c. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026d. The cut-agnostic division theorem: why every open problem is exactly its proved part and its unicorn. Zenodo. doi:10.5281/zenodo.22954862.
- Islam, M. F. 2026e. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026f. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026g. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026h. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026i. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Islam, M. F. 2026j. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026k. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapient/Trisduction, GitHub.
- Kodaira, K. and D. C. Spencer. 1953. Divisor class groups on algebraic varieties. *Proceedings of the National Academy of Sciences* 39: 872--877.
- Lefschetz, S. 1924. *L'Analysis situs et la géométrie algébrique*. Paris: Gauthier-Villars.